

When FPGA Meets Dataflow Analysis: An Explorative Step

FANG WEI*, Nanjing University, China

QINLIN CHEN*, Nanjing University, China

NAIREN ZHANG, Nanjing University, China

JIACAI CUI, Nanjing University, China

TIAN TAN[†], Nanjing University, China

ZHIQIANG ZUO, Nanjing University, China

YUE LI[†], Nanjing University, China

Set-based (a.k.a. bit-vector-based) dataflow analysis is a fundamental building block for many static analysis tasks, and significant effort has been devoted to accelerating it. Existing acceleration approaches address the problem from a *software* perspective, leveraging various general-purpose computing platforms, such as single- and multi-core CPUs, GPUs, and distributed systems. In contrast, a *hardware-centric* approach—designing specialized hardware that directly accelerates dataflow analysis—remains unexplored.

Motivated by this gap and out of pure research curiosity, we conduct a preliminary exploration of designing specialized hardware for dataflow analysis using FPGAs, which are highly customizable and well suited for rapidly prototyping domain-specific hardware. As a first step toward hardware-accelerated dataflow analysis, we focus on the widely used intra-procedural dataflow analysis. However, we find that designing specialized hardware even for this setting is already challenging: a straightforward FPGA implementation of the classical worklist algorithm is *infeasible*, because its space complexity grows superlinearly with procedure size, quickly exhausting the FPGA's limited high-speed on-chip memory when analyzing large procedures.

To address this challenge, we introduce FPGAFlow, a specialized hardware design for dataflow analysis that (1) overcomes the spatial infeasibility challenge by leveraging the distributivity of set-based dataflow analysis to achieve linear spatial scalability, and (2) accelerates analysis through hardware-specific parallelism—pipelining with data forwarding and BRAM partitioning and replication.

We evaluate FPGAFlow on diverse and popular real-world Java projects (averaging 32.5k GitHub stars) using two representative dataflow analyses—live variables and reaching definitions—and compare it against their software implementations in a state-of-the-art Java static analyzer Tai-e. In terms of correctness, FPGAFlow produces exactly the same analysis results as Tai-e, amounting to 75 billion bits. In terms of acceleration, even on a modest Xilinx Zynq-7020 FPGA (55 MHz), FPGAFlow achieves an average speedup of 15.45× for live variables and 12.32× for reaching definitions compared with Tai-e running on a server-grade CPU (2.20 GHz to 3.00 GHz). We hope this work offers useful insights toward future FPGA-accelerated static analysis.

CCS Concepts: • **Hardware** → **Hardware accelerators**; • **Software and its engineering** → **Automated static analysis**.

Additional Key Words and Phrases: FPGA Acceleration, Static Analysis, Dataflow Analysis

*Both authors contributed equally to this paper.

[†]Corresponding author.

Authors' Contact Information: Fang Wei, fangwei@smail.nju.edu.cn, Nanjing University, Nanjing, China; Qinlin Chen, qinlinchen@smail.nju.edu.cn, Nanjing University, Nanjing, China; Nairen Zhang, nairen@smail.nju.edu.cn, Nanjing University, Nanjing, China; Jiakai Cui, jiacaicui@smail.nju.edu.cn, Nanjing University, Nanjing, China; Tian Tan, tiantan@nju.edu.cn, Nanjing University, Nanjing, China; Zhiqiang Zuo, zqzuo@nju.edu.cn, Nanjing University, Nanjing, China; Yue Li, yueli@nju.edu.cn, and all authors are also affiliated with the State Key Laboratory of Novel Software Technology, Nanjing University, Nanjing, China.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/10-ART376

<https://doi.org/10.1145/3839508>

ACM Reference Format:

Fang Wei, Qinlin Chen, Nairen Zhang, Jiakai Cui, Tian Tan, Zhiqiang Zuo, and Yue Li. 2026. When FPGA Meets Dataflow Analysis: An Explorative Step. *Proc. ACM Program. Lang.* 10, OOPSLA2, Article 376 (October 2026), 27 pages. <https://doi.org/10.1145/3839508>

1 Introduction

Set-based (a.k.a. bit-vector-based) dataflow analysis serves as a fundamental building block for many static analysis tasks, including bug detection [32, 35], security analysis [3, 6, 19, 23], code optimization [1], and program understanding [22, 34, 44]. Accordingly, the research community has devoted significant effort to accelerating such analyses. Existing acceleration approaches commonly address the problem from a *software* perspective, leveraging various general-purpose computing platforms, such as single- and multi-core CPUs [31, 33, 42, 50], GPUs [4, 45], and distributed systems [37, 49].

In contrast, a *hardware-centric* approach—designing domain-specific hardware to directly accelerate dataflow analysis—remains unexplored, despite its demonstrated success in domains such as deep learning and cloud computing [5, 12, 29, 30, 47]. Unlike software-centric efforts, which adapt dataflow analysis algorithms to general-purpose computing platforms, a hardware-centric approach allows for more specialized use of hardware resources.

Motivated by this gap and out of pure research curiosity, we conduct a preliminary exploration of designing specialized hardware for dataflow analysis. We choose to base our design on FPGAs since they are highly customizable and convenient for rapid prototyping of domain-specific hardware designs. As a first step toward hardware-accelerated dataflow analysis, we focus on intra-procedural dataflow analysis, which is widely used [1].

However, our exploration reveals that designing specialized hardware—even for intra-procedural dataflow analysis—is already challenging. The primary obstacle is making the analysis *spatially feasible* within the tight high-speed on-chip memory constraints of FPGAs. We found that a straightforward FPGA implementation of the classical worklist algorithm makes large procedures unanalyzable on-chip. Software algorithms typically assume abundant memory, allocating storage for dataflow fact sets at every CFG node. This incurs $O(|CFG| \times |D|)$ memory, where $|D|$ is the fact domain size and both terms generally grow with procedure size. While this $O(|CFG| \times |D|)$ memory requirement is manageable on general-purpose platforms, it can quickly exhaust the limited high-speed on-chip memory of FPGAs. As a result, large procedures must fall back to a software implementation, losing the potential acceleration offered by the FPGA.

To address this challenge, we introduce **FPGAFlow**, a specialized hardware design for dataflow analysis that proceeds in two stages: making large-procedure analysis feasible on the FPGA and improving acceleration speedup. Specifically:

- (1) For *spatial feasibility*, we leverage the distributivity of dataflow merge and transfer functions (Theorem 1) to decompose fact propagation into fixed-size chunks and restructure the analysis so that on-chip memory requirements scale linearly with $|CFG|$.
- (2) For better *temporal efficiency*, we adopt a pipelined design with data forwarding and BRAM partitioning and replication, maximizing hardware parallelism and throughput.

Currently, **FPGAFlow** is specialized for set-based dataflow analyses whose flow functions satisfy distributivity. Analyses outside this scope would likely require different hardware designs. This limitation reflects a fundamental trade-off in domain-specific acceleration: performance gains are achieved by trading generality in favor of specialization. Despite this boundary, we hope that our exploration of leveraging algebraic properties and hardware-software co-design to accelerate program analysis can inspire the development of specialized accelerators for a broader range of

analyses. We exemplify this direction by illustrating a high-level idea for adapting FPGAFlow to constant propagation, a representative non-set-based dataflow analysis, in Section 4.4.2.

For better usability, FPGAFlow provides a pluggable Ethernet interface, enabling existing software static analyzers to leverage its pure hardware-based acceleration by implementing lightweight driver code to format and transmit the program-under-analysis data. As a demonstration, we integrated FPGAFlow with Tai-e [38], a state-of-the-art Java static analyzer, implementing the driver in about 100 lines of code to accelerate its dataflow analysis.

To validate the correctness and acceleration effect of FPGAFlow, we evaluate it using two representative dataflow analyses—live variables [1, 17] and reaching definitions [1, 2]—in comparison to their pure software implementations in Tai-e. Our benchmark suite for evaluation consists of real-world open-source Java projects [8–11, 14–16, 26, 27, 40, 41, 46, 48] (averaging over 32.5k GitHub stars) across diverse domains, including foundational frameworks, distributed middleware, development tools, and business applications. The experimental results are encouraging:

- *Correctness.* Across all evaluated real-world projects, FPGAFlow produces exactly the same analysis results, amounting to over 75 billion bits that represent the final dataflow facts.
- *Acceleration.* Note that our FPGA-based solution runs on a modest FPGA (approx. \$130, 28nm, operating at 55 MHz), while the software baseline runs on a high-end CPU (approx. \$600, 14nm, 2.20 GHz to 3.00 GHz). Despite this disparity in platform capability, FPGAFlow still achieves an average speedup of 15.45 $\times$ for live variables and 12.32 $\times$ for reaching definitions.

To help deploy FPGAFlow across different FPGA boards with their own on-chip resource budgets, we further studied how to choose a key hyperparameter, the *data-fact width*, which determines the accelerator’s degree of parallelism when processing dataflow bitvectors. We establish a theoretical upper bound (Theorem 2) on the data-fact width under a given memory budget: below this bound, increasing the width improves parallelism and thus performance. Beyond the bound, however, a larger width causes more procedures to overflow on-chip memory and fall back to software execution, which can negate the speedup. We conduct a study to characterize this trade-off, showing that exceeding the bound can yield marginal speedup gains. Therefore, the best performance is typically achieved when the data-fact width slightly exceeds, but remains close to, the bound.

In summary, this work makes the following contributions:

- We present FPGAFlow, the first specialized FPGA-based hardware design for accelerating dataflow analysis, a direction previously unexplored in program analysis research. FPGAFlow overcomes the spatial-feasibility barrier, enabling large procedures to be analyzed entirely on chip, and exploits hardware parallelism to improve temporal efficiency.
- We conduct a preliminary experiment and the results show that FPGAFlow outperforms a state-of-the-art software static analyzer, achieving average speedups of 15.45 $\times$ for live variables and 12.32 $\times$ for reaching definitions, showcasing the potential of FPGA-accelerated static analysis.
- We implement an end-to-end system spanning the host-side analysis interface and FPGA hardware, and we release it as open source [43] to support further research and development in hardware-accelerated static analysis.

We hope this work presents an interesting exploration into FPGA-accelerated static analysis. Although within the intra-procedural scope studied here, the speedup (15.45 $\times$ for live variables and 12.32 $\times$ for reaching definitions) may not be particularly attractive for some real deployments, we believe it still represents a concrete and feasible first step that exposes the potential of FPGA-acceleration in static analysis. By exploiting both analysis-specific and hardware-specific characteristics, FPGA accelerators can enable more efficient use of hardware resources than conventional general-purpose processors and greatly reduce analysis execution time.

Looking beyond the specific prototype explored in this paper, we envision that future FPGA acceleration could benefit more computationally intensive analyses that scale poorly even on modern multi-core CPUs. For example, a next step is perhaps designing FPGA accelerators for inter-procedural dataflow analysis [37], where the dominant challenge is scaling fact propagation to whole-program CFGs, and a possible solution could be designing efficient coordination across limited high-speed on-chip memory and abundant off-chip memory, or scaling out with FPGA arrays, to sustain throughput at whole-program scale. Furthermore, the FPGA acceleration approach may also provide an opportunity to accelerate the well-known expensive context-sensitive analyses [20, 21, 25, 36, 39], which remain challenging to scale due to their high computational complexity. Ultimately, we envision a future in which developers can accelerate static analyses simply by plugging an inexpensive, off-the-shelf FPGA accelerator card into their machines. Such accelerators could shorten developer feedback cycles, improve productivity, and make more sophisticated static analyses practical for routine use during software development.

2 FPGA-Accelerated Dataflow Analysis: System Overview

FPGA acceleration introduces a system that spans the host software stack and the hardware on the accelerator card. Figure 1 shows the complete acceleration system developed in this work. Before diving into the design specifics, and considering that FPGA hardware acceleration may be unfamiliar to many readers, this section provides an overview of our acceleration system. This overview provides the context for understanding the accelerator designs in Sections 3 and 4. Section 3 introduces a straightforward accelerator that directly maps the classical dataflow analysis algorithm onto an FPGA, serving as a motivating example to highlight the challenges of FPGA-based acceleration under constrained hardware resources. Section 4 then builds on this foundation, presenting our optimized design that addresses these challenges.

Figure 2 illustrates the system overview, partitioned into two distinct domains: a software-based host system and a hardware-based accelerator card. These two components are bridged by a high-speed I/O interface (e.g., Ethernet and PCIe), and any software static analyzer on the host can benefit from the hardware-based acceleration by implementing minimal driver code to format and transmit the program data required for analysis. To intuitively illustrate the acceleration process, we describe the system components following the sequence of data transfers (Steps 1–15) depicted in Figure 2.

First, the host system comprises a software static analyzer and the host-side memory. The software static analyzer is tasked with parsing the program under analysis and preparing the data required for FPGA-accelerated dataflow analysis. Specifically, it retrieves the program under analysis from the memory (Step 1), preprocesses it into initial data facts and auxiliary analysis information (e.g., control-flow graph), and transmits them to the accelerator card via a high-speed I/O interface (Step 2).

Second, on the accelerator card, the processing system (PS) functions as the local control hub between the host system and the accelerator kernel that performs the core computations. Specifically, the PS receives the transmission (Step 3) and offloads the data to the accelerator card's on-board SDRAM memory (Step 4). It then forwards the required data from the on-board memory to the accelerator kernel via an internal data bus (e.g., AXI) and starts the acceleration (Steps 5–7). Note



Fig. 1. The complete acceleration system developed in this work, in which the FPGA is connected to the host via an Ethernet cable.

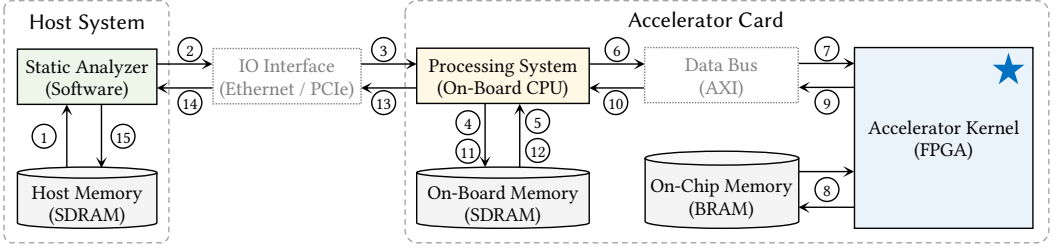


Fig. 2. Overall architecture of FPGAFlow. It consists of a *host system* and an *accelerator card* connected via a high-speed *input-output (I/O) interface*. The host reuses an existing software static analyzer to preprocess the program and generate inputs (such as the control-flow graph) for the accelerator. The accelerator features a hybrid design where a processing system handles on-board control and external communication, while the customizable *accelerator kernel*, implemented with the *field programmable gate array* (FPGA), executes our core accelerated dataflow analysis algorithm. The arrows indicate the direction of data transfer between components, and the accompanying numbers denote the order in which the data transfers occur.

that we strategically retain the software static analyzer on the host rather than migrating it to the PS. This is because the host's abundant memory and powerful CPU are better suited for executing code with complex logic for preparing the data for acceleration, while the PS primarily acts as a data relay station.

Third, connected to the PS via the AXI data bus, the accelerator kernel serves as a customizable computing unit executing the dataflow analysis algorithms. During execution, it leverages on-chip BRAM to cache intermediate analysis states (Step 8). This is because BRAM provides significantly lower access latency than the on-board SDRAM, although its capacity is limited. Efficient design of the accelerator kernel and optimal utilization of the constrained on-chip memory are central to achieving effective FPGA-based acceleration. We detail these aspects in Section 3 and Section 4.

Finally, upon completion, the accelerator kernel writes the computation results back to the on-board memory (Steps 9–11). Subsequently, the PS retrieves the analysis results and transfers them back to the host system (Steps 12–15).

3 First Step: A Straightforward Accelerator

Presenting the full set of FPGAFlow design details for FPGA-accelerated dataflow analysis upfront would make it difficult for readers to see why each technique is necessary, due to a lack of motivation and a concrete baseline. To address this, we adopt a progressive approach: we begin with a deliberately straightforward accelerator that maps the classic software worklist algorithm for dataflow analysis to hardware in a one-to-one manner. This straightforward design is also where our exploration started—by first constructing the most direct translation, we obtain an easy-to-follow baseline and learn which bottlenecks dominate on an FPGA, which in turn guides the optimizations introduced in Section 4.

This idea is illustrated in Figure 3, which places the software worklist algorithm (left) alongside its direct hardware counterpart (right) and highlights the correspondence between algorithmic steps and hardware units (arrows). We organize this section around Figure 3 as follows: Section 3.1 revisits the worklist algorithm (left); Section 3.2 explains the direct mapping from algorithmic stages to hardware units (arrows) and the resulting hardware execution flow (right); and Section 3.3 summarizes the challenges revealed by this straightforward mapping.

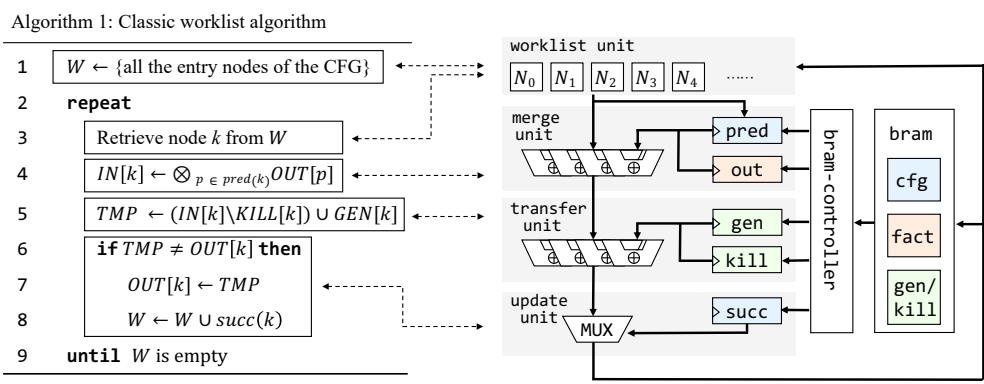


Fig. 3. The conceptual correspondence between the classic worklist algorithm for dataflow analysis (on the left) and the execution flow (simplified for ease of understanding) of our straightforward accelerator (on the right), which serves as a first-step design toward FPGAFlow. The mapping of the stages in the worklist algorithm to physical components in our FPGA accelerator is depicted by bidirectional dashed arrows.

3.1 Recap: The Software Worklist Solver

Worklist-based solvers are widely adopted as the standard implementation technique for dataflow analysis [1]. We briefly recap the algorithmic skeleton shown on the left side of Figure 3, which is a slightly modified version of the algorithm in Figure 9.14 of the classic compilation textbook [1]. Specifically, we omit the initialization phase and treat each statement as a basic block. Note that in this paper, we focus on forward dataflow analyses for ease of presentation, but the same principles apply to backward analyses with appropriate adjustments to the direction of CFG traversal.

The solver maintains a worklist of CFG nodes to process (initialized with entry nodes) and iterates until the worklist is empty. Each iteration removes a node (Line 3), computes its input fact by applying the meet operator to incoming facts from predecessors (Line 4), applies the transfer function (e.g., gen/kill) to obtain a new output fact (Line 5), and enqueues successors whenever a fact changes (Lines 6–8). When the worklist becomes empty (Line 9), a fixed point has been reached, yielding the final dataflow analysis result.

3.2 Straightforward Hardware Mapping and Execution Flow

Guided by the correspondence in Figure 3 between worklist stages and hardware units, this section describes our straightforward hardware accelerator that directly implements the classic worklist algorithm in Section 3.1. We first use the right half of Figure 3 to present an abstract view of the accelerator that focuses on the execution flow and omits micro-architectural details. We then refer to Figure 4, which shows the detailed circuit used in our straightforward accelerator, to explain how this abstract flow is realized in hardware. For reference, the hardware components described in this section correspond to the accelerator kernel and BRAM blocks in the system overview (Figure 2).

As indicated by the dashed arrows in Figure 3, the one-to-one mapping is realized as follows:

- **Worklist Unit:** Corresponds to worklist initialization and task retrieval (“pop one node”) (from Lines 1 and 3). It maintains the FIFO worklist, which stores CFG node indices; in each iteration, it outputs a node index to the Merge Unit and accepts the successor nodes from the Update Unit (discussed later) for subsequent processing.
- **Merge Unit:** Corresponds to computing the IN fact of a given node via the meet operator (from Line 4). Given a node index from the Worklist Unit, it uses the pred module to fetch the node’s

predecessor indices from BRAM (via the shared bram-controller), then uses the out module to read the corresponding predecessor *OUT* facts. Finally, it performs the meet operation on these *OUT* facts and forwards the resulting *IN* to the Transfer Unit. Note that dataflow facts, represented as sets in the software implementation, are encoded as bitvectors in the accelerator kernel; Section 4.1.1 details how the accelerator handles these bitvectors efficiently.

- *Transfer Unit*: Corresponds to applying the node transfer function (e.g., *GEN/KILL*) (from Line 5). It uses the gen and kill modules to fetch the node's *GEN* and *KILL* sets, also encoded as bitvectors, from BRAM through the bram-controller. It then combines them with *IN* from the Merge Unit using bitwise set operations (i.e., union and difference), and produces a temporary output fact (*TMP*) for the Update Unit.
- *Update Unit*: Corresponds to checking for convergence, updating *OUT*, and propagating changes (from Lines 6–8). It compares *TMP* from the Transfer Unit against the *OUT* fact of the current node (previously read via the out module of the Merge Unit from BRAM); if they differ, it writes back the updated *OUT* to BRAM and uses the succ module to fetch successor indices, which are sent to the Worklist Unit.

At the top level, the repeat/until loop (Lines 2 and 9 in Figure 3) is naturally driven by the FIFO worklist: the loop continues as long as the worklist is non-empty, and terminates when it becomes empty.

Figure 4 shows a refined, circuit-level implementation of the abstract flow in Figure 3. The accelerator is implemented as a synchronous circuit with multi-cycle execution for each analysis iteration. We describe this refined design in the same order as above, following a single analysis iteration as it propagates sequentially through the Worklist, Merge, Transfer, and Update units.

The iteration begins when a node index is dequeued from the FIFO in the *Worklist Unit*. The rd_controller is responsible for dispatching this node to the downstream logic. The design of the Worklist Unit requires special attention due to the interaction between successor propagation and the accelerator's single-node processing model. Because the *Update Unit* may enqueue multiple successor nodes that require further iterations in a single iteration, the FIFO is designed to support enqueueing and dequeuing multiple node indices at once. However, the accelerator circuit can process only one node per iteration. Therefore, if a dequeue operation returns more than one node index, the extra indices are stored in a skid_buffer. While the skid_buffer still contains unprocessed node indices, the FIFO does not perform another dequeue operation.

Once dispatched, the node index enters the *Merge Unit*, which gathers the information required to compute its input fact (*IN*). Using this node index, the fetch_pred_controller queries the CFG representation stored in BRAM—via the bram_controller—to retrieve the indices of all predecessor nodes. Using these predecessor indices, the fetch_out_controller reads the corresponding predecessor output facts (*OUT*), which are immediately supplied to the merge_function. The

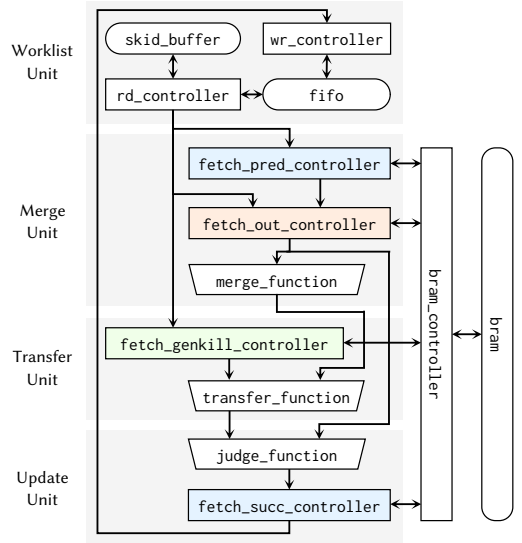


Fig. 4. Circuit-level implementation of the straight-forward accelerator in Figure 3.

`merge_function` performs the meet operation over these predecessor *OUT* facts and produces the input fact *IN* for the current node. The `fetch_out_controller` also retrieves the current node's previously stored *OUT* value from BRAM, which is preserved for the comparison performed later in the Update Unit.

The computed *IN* is then forwarded to the Transfer Unit. Using the same node index dispatched earlier by the `rd_controller`, the `fetch_genkill_controller` retrieves the node-specific *GEN* and *KILL* sets from BRAM. Together with *IN*, these sets are processed by the `transfer_function`, which implements the transfer function $TMP \leftarrow (IN - KILL) \cup GEN$ using bitwise hardware operations to generate the new temporary output fact.

Finally, *TMP* reaches the Update Unit. The `judge_function` compares *TMP* against the previously fetched *OUT* value of the current node (obtained earlier via the `fetch_out_controller`). If the two facts are identical, the iteration terminates for this node. Otherwise, the `fetch_suc_controller` writes back the updated *OUT*, and retrieves all successor node indices of the current node. These successors are passed to the *Worklist Unit*, where the `wr_controller` enqueues them into the FIFO, allowing the effect of the updated fact to propagate through the CFG.

3.3 Challenges Revealed by the Straightforward Mapping

Although the straightforward design faithfully reflects the software worklist algorithm in hardware, its limitations become apparent when considering larger procedures. These limitations manifest along two dimensions: *spatial infeasibility* and *temporal inefficiency*. In the following, we explain how both arise directly from the straightforward mapping.

The first limitation is spatial infeasibility: the straightforward FPGA mapping fails to analyze large procedures due to limited high-speed on-chip memory (i.e., BRAM). In this design, each CFG node maintains its own data facts (i.e., *OUT*) in BRAM, and the CFG structure is also stored in BRAM. As procedure size increases, both the number of CFG nodes and the width of each data fact grow, causing the total BRAM requirement to scale with the product of these two factors. Because BRAM resources are limited, this storage scheme can quickly exhaust available capacity when analyzing a large procedure. As a result, the straightforward design does not scale well without rethinking how data facts and control-flow information are represented in hardware.

Even for procedures that fit within BRAM, temporal inefficiency emerges as a second major concern. The straightforward accelerator processes exactly one node per iteration. Task retrieval, merging, transfer, and update are executed in a strictly serialized flow. While this mirrors the structure of the software worklist solver, it prevents parallelizing the processing of multiple nodes. Consequently, only part of the hardware datapath is actively utilized at any given time, leaving other functional units idle. Given that FPGAs generally operate at lower clock frequencies than high-end CPUs, the lack of effective parallelism directly constrains achievable speedup in the straightforward design.

Taken together, these spatial and temporal limitations indicate that a direct software-to-hardware translation is insufficient for achieving scalable and efficient acceleration. They motivate our optimizations presented in the next section.

4 The FPGAFlow Approach

As discussed in Section 3.3, a straightforward FPGA implementation of the classical worklist algorithm suffers from both spatial infeasibility and temporal inefficiency. Large procedures frequently exhaust the FPGA's limited high-speed on-chip memory, preventing acceleration; even for those that fit in memory, the FPGA's typically lower clock frequency (relative to CPUs) can limit the achievable speedups. To address these challenges, we introduce a series of optimizations that build on the straightforward FPGA implementation. Specifically, Section 4.1 shows how we exploit the

distributivity of set-based dataflow analyses to make large procedures analyzable on an FPGA, and Section 4.2 presents how we leverage hardware-specific parallelism to further accelerate the analysis. Finally, Section 4.3 describes how a software static analyzer can implement a lightweight driver to leverage FPGAFLOW for accelerating dataflow analysis.

4.1 Enabling Spatial Feasibility for Large Procedures

For each procedure under dataflow analysis, a straightforward FPGA implementation of the classical worklist algorithm stores both the CFG and the per-node bitvector facts (i.e., *OUT*) in on-chip memory (i.e., BRAM). As discussed in Section 3.3, this design makes BRAM demand grow superlinearly with procedure size, since both the number of CFG nodes and the width of the bitvectors increase as procedures become larger. Consequently, large procedures can quickly exceed the limited high-speed on-chip memory available on FPGAs and cannot be analyzed on chip.

To address this challenge, we leverage the *distributivity of data-fact operators* (Section 4.1.1) to split oversized per-node data facts into chunks for processing. This chunk-wise design avoids keeping full-width data facts on chip and reduces BRAM demand to scale linearly with the CFG size. In addition, we *restructure the CFG* (Section 4.1.2) to make its on-chip representation more compact. Together, these optimizations enable FPGAFLOW to analyze almost all procedures in our evaluation, except for rare cases where a procedure's CFG is too large to fit in on-chip memory.

4.1.1 Leveraging the Distributivity of Data-Fact Operators. In the set-based dataflow analysis, each node maintains data facts (e.g., *OUT*). FPGAFLOW represents these facts as bitvectors, where each bit corresponds to a program property. For example, in reaching definitions, a bit can represent whether a particular definition reaches the current CFG node. As the analyzed procedure grows in size, the number of properties increases and the fact bitvectors become wider. Storing full-width facts for every CFG node can therefore exceed the FPGA's on-chip BRAM capacity.

Our key observation is that the merge (meet) and transfer operations in our worklist-style set-based analysis satisfy distributivity (Theorem 1). This allows us to split a wide data-fact bitvector into fixed-size chunks, process them sequentially, and concatenate them at the end to reconstruct the full bitvector, which is equivalent to processing the full bitvector at once.

Concretely, we initialize the full data-fact bitvector for each node in host memory, which offers higher capacity but lower performance than on-chip BRAM. We partition each bitvector into w -bit chunks, where w is a tunable hyperparameter called the *data-fact width*. For each chunk index i , we load into BRAM the i -th chunk of the *OUT* facts for every CFG node. We then run the hardware worklist iterations on this set of i -th chunks until it converges. After the i -th chunk converges, we write the resulting i -th chunk for each node back to the host memory. We repeat this process for $i + 1$. After all chunk indices are processed, the host memory holds the per-chunk results, which are concatenated to reconstruct the complete facts for each node.

Theorem 1 (Distributivity of Data-Fact Operators). *Let F be an operator that maps k input bitvectors $\mathbf{x}_1, \dots, \mathbf{x}_k$ to an output bitvector. Fix a chunk width w . For a bitvector $\mathbf{x}$, let $\mathbf{x}[i]$ denote its i -th w -bit chunk. We say F is distributive if, for any inputs $\mathbf{x}_1, \dots, \mathbf{x}_k$ and any chunk index i ,*

$$(F(\mathbf{x}_1, \dots, \mathbf{x}_k)) [i] = F(\mathbf{x}_1[i], \dots, \mathbf{x}_k[i]).$$

Under the standard bitvector encoding of sets, the merge (meet) and transfer operators used in the worklist-style set-based dataflow analysis satisfy this property.

PROOF SKETCH. It is straightforward to verify that both the merge (meet) operator and the transfer operator (which contains set difference and set union) satisfy the distributivity property, and therefore the theorem follows. $\square$

Note that a larger w (i.e., a wider data-fact width) processes more data facts per iteration and thus reduces the number of chunks, improving runtime performance. However, it also increases the per-node on-chip storage footprint and reduces the maximum procedure size that fits in BRAM. In Section 5, we provide empirical results and a theoretical model (Section 5.2) to guide the choice of w under different on-chip memory budgets.

4.1.2 Restructuring the Control Flow Graph. In software, a CFG is typically represented using variable-length data structures such as adjacency lists, allowing each node to have a flexible number of predecessors and successors. Such representations require extra metadata (e.g., list lengths) and rely on indirect memory accesses (e.g., pointer dereferences) to retrieve neighboring nodes. While this approach is efficient in software, where memory is plentiful and CPUs excel at handling indirect accesses, it is ill-suited for FPGAs. On an FPGA, limited on-chip BRAM capacity and less sophisticated control logic make variable-length structures inefficient to store and access. This inefficiency is particularly problematic in throughput-oriented accelerator designs.

A common hardware-friendly approach is to preallocate, for each node, a fixed number of successor and predecessor slots in on-chip memory. This makes memory accesses simple and efficient (e.g., the i -th successor can be read from a fixed location), but it introduces a representability-efficiency trade-off. If we reserve too few slots per node (e.g., 2), nodes with more than two successors (e.g., a switch dispatch node) or predecessors cannot be represented in this memory layout; if we reserve more slots (e.g., 4 or more), many reserved entries remain unused because most nodes have only one or two edges, wasting BRAM.

To address this issue, we propose to restructure the control flow graph on the host CPU. For any node with three or more predecessors or successors, we introduce intermediate nodes to rewrite the local CFG structure into an equivalent form where each node has at most two predecessors and at most two successors. Figure 5 illustrates a restructuring example of a switch statement. Initially, the switch node in the control flow graph directly connects to three successors. Through the introduction of temporary nodes, we successfully limit the number of successors for each node to two.

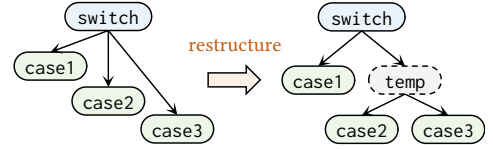


Fig. 5. An example of CFG restructuring.

Through this restructuring, we only need to reserve storage space for exactly two predecessors and two successors per node on the FPGA. This keeps successor/predecessor accesses fast and simple on the FPGA—each entry is read by directly indexing a fixed per-node slot—while avoiding the allocation of many unused slots and improving FPGA BRAM utilization. Although restructuring the CFG takes additional time on the host CPU, it typically touches only a small number of CFG nodes, since nodes with more than two predecessors or successors are relatively rare in typical CFGs; thus, the overall overhead is modest.

4.2 Improving Temporal Efficiency via Hardware Parallelism

While the spatial optimizations above make it feasible to analyze large procedures on an FPGA, overall performance still depends on sufficient execution throughput. To improve throughput, we introduce hardware-specific parallelism—a pipelined microarchitecture that overlaps the processing of different nodes (Section 4.2.1). This pipeline, however, also increases the demand for low-latency data access and high memory bandwidth. Accordingly, we incorporate two pipeline-driven optimizations. First, we add data forwarding paths in the pipeline to prevent fetching stale data during BRAM read-after-write hazards, thereby avoiding delayed convergence, which ultimately improves iteration efficiency (Section 4.2.2). Second, we exploit BRAM partitioning and replication to

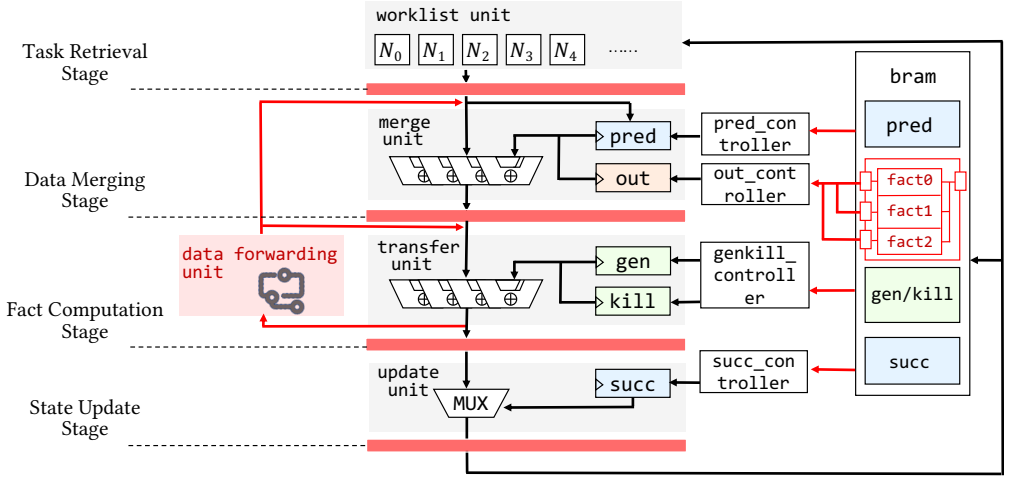


Fig. 6. The architecture of FPGAFlow. Modules highlighted in red correspond to the microarchitectural optimizations in Section 4.2. The design is centered around a pipelined processing kernel, and is further supported by a data forwarding path and a partitioned/replicated BRAM subsystem.

support concurrent memory accesses across pipeline stages, increasing effective memory bandwidth (Section 4.2.3). Figure 6 presents a high-level view of our final accelerator microarchitecture. The components highlighted in red correspond to the optimizations introduced in this section. In the following subsections, we explain each optimization in turn and use Figure 6 to locate the highlighted modules in the overall design.

4.2.1 Pipelined Design. Compared to the straightforward accelerator in Section 3, our final design adopts an explicit pipelined design to overlap the processing of different nodes. In the straightforward design, a CFG node is processed by the *Worklist*, *Merge*, *Transfer*, and *Update* units in a strictly serialized manner, as shown on the right side of Figure 3. Our key change is to insert pipeline registers between these major units, as indicated by the red horizontal lines in Figure 6. These registers define stage boundaries and decouple adjacent units so that multiple nodes can be processed concurrently. This helps keep the units busy and can improve the accelerator’s sustained throughput (e.g., nodes per cycle), rather than processing one node end-to-end.

However, pipelining the worklist solver is not a drop-in optimization. Overlapping stages introduces data hazards (e.g., read-after-write on *OUT* facts) and increases concurrent pressure on BRAM ports and bandwidth, which can stall the pipeline and erode throughput. We address these issues with data forwarding (Section 4.2.2) and BRAM partitioning/replication (Section 4.2.3), as described next. For ease of exposition, we group the pipeline into four major stages based on the dominant work performed in each unit: *Task Retrieval*, *Data Merging*, *Fact Computation*, and *State Update*, as shown in Figure 6.

4.2.2 Data Forwarding. In our pipeline, the *State Update* stage writes the newly computed *OUT* fact of a node back to BRAM. Meanwhile, the *Data Merging* stage reads predecessor *OUT* facts from BRAM to compute the next node’s *IN*. Because these stages can execute in parallel, the design can encounter a read-after-write hazard. That is, the merging stage may read a stale *OUT* value of a node before the update stage has written back its recently updated fact.

The stale *OUT* value delays algorithmic convergence, although the monotonicity of the analysis guarantees correctness even when this hazard is ignored. To accelerate algorithmic convergence,

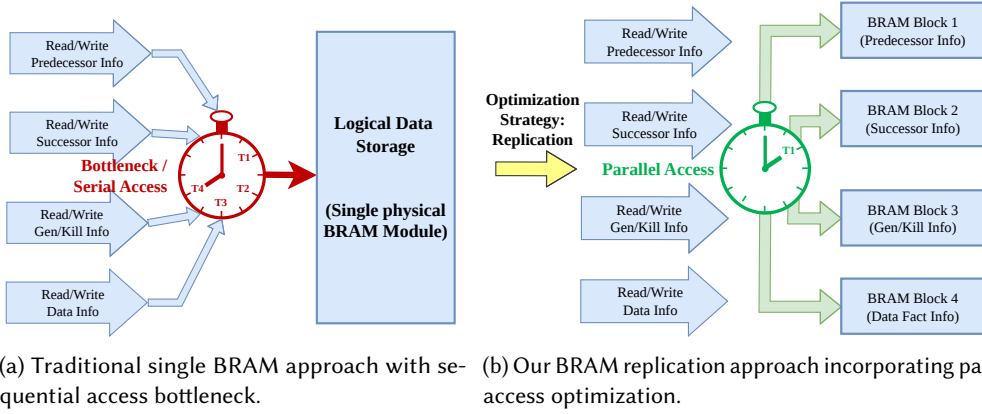


Fig. 7. Overview of the BRAM replication strategy used to eliminate memory bottlenecks. By partitioning the storage into independent BRAM blocks, the architecture enables concurrent memory access patterns.

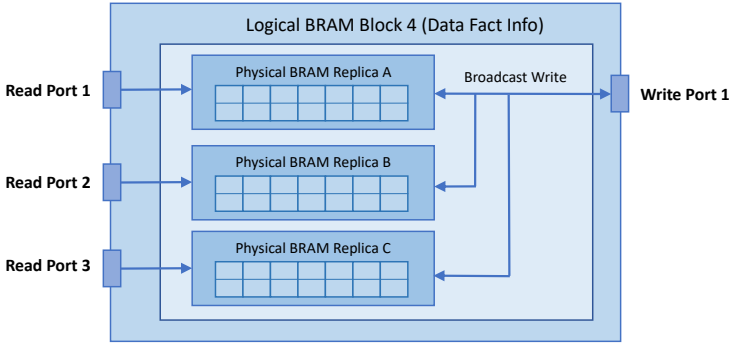


Fig. 8. Internal design of the replicated BRAM block (specifically for *Data Fact Info*). To overcome the read-port limitation, the physical storage is replicated multiple times (e.g., Replicas A through C). A *Broadcast Write* mechanism updates all replicas simultaneously to maintain data coherency, while multiple independent read ports enable parallel data retrieval, creating a *Single-Write-Multi-Read* interface.

we adopt a *Data Forwarding* mechanism. Specifically, we add a bypass path (highlighted by the red arrows on the left side of Figure 6) that forwards the freshly produced *OUT* fact from *Fact Computation* directly to *Data Merging* when needed. This enables the merge stage to use the most up-to-date facts without waiting for BRAM write-back.

4.2.3 BRAM Partitioning and Replication for Memory Bandwidth. Our pipeline design requires simultaneous access to multiple data types in a single cycle. Specifically, the system must concurrently read predecessor, successor, and gen/kill information from BRAM. However, an individual FPGA BRAM block exposes only a small number of access ports. In the common simple dual-port configuration, this effectively means one write port and one read port per cycle, which is often insufficient for our concurrent read requests. As illustrated on the left side of Figure 7, with a single BRAM block these reads cannot be served in parallel and must be issued over multiple cycles. Consequently, fetching predecessor, successor, gen/kill, and data fact information can take several cycles (T1 to T4), which stalls the pipeline and reduces throughput.

To address this bottleneck, we adopt a BRAM partitioning strategy. As shown on the right side of Figure 7, we split the storage by data type and place each type in a separate BRAM block. Importantly, this does *not* make a single BRAM provide more read ports. Rather, it gives the design *multiple* BRAMs to read from. Since each BRAM can serve one read per cycle, the hardware can issue multiple reads in the same cycle as long as they go to different BRAMs. This matches our common access pattern, which needs one item from each data type per cycle. As a result, predecessor, successor, gen/kill, and data-fact information can be fetched concurrently in a single cycle (T1).

Moreover, partitioning across data types does not help when multiple concurrent reads target the same logical structure. In particular, the output facts stored in the *Data Fact Info* block still face a read-bandwidth bottleneck, since the hardware may need multiple distinct reads in the same cycle.

To address this, we implement an intra-block replication mechanism that trades additional BRAM capacity for higher effective read bandwidth. Figure 8 details the design. For the logical *Data Fact Info* block, we instantiate multiple physical BRAM replicas (e.g., Replicas A through C). Reads are served from different replicas in parallel, allowing multiple distinct read requests in the same cycle. To keep the replicas consistent, we design a *Broadcast Write* interface that sends each write request to all replicas in the same cycle.

In Figure 6, this optimization corresponds to the right-hand-side BRAM module, whose differences from Figure 3 are highlighted in red. The module contains four type-partitioned subblocks, which enable parallel reads of data from different types. This parallelism is visually represented in the figure by the red arrows originating from the left side of the BRAM. It also replicates the *Data Fact Info* storage into multiple BRAM copies to enable parallel reads of the same data type. Together, these changes improve effective memory bandwidth and help sustain pipeline throughput.

4.3 Integrating Static Analyzers through a Lightweight Driver

For better usability, FPGAFlow offers a pluggable, network-based interface. Once the host running an existing software static analyzer connects to the FPGA over Ethernet, the analyzer can leverage FPGAFlow's hardware-based acceleration by adding only lightweight driver code to drive the accelerator. We describe this interface from both the host-software and FPGA perspectives.

On the host side, the integration process is straightforward. For each procedure, the host first sends the analysis configuration, which specifies the analysis direction (forward/backward) and the initial fact state (a single variable denoting an all-zero or all-one initialization). It then sends the restructured CFG structure (Section 4.1.2) and the corresponding chunked gen/kill facts (Section 4.1.1), which are extracted by the software-based static analyzer. All these data components are transmitted to the FPGA as standard TCP packets. Because modern static analyzers are typically developed in high-level programming languages, formatting and sending this data over a socket requires minimal engineering effort. For example, we integrated FPGAFlow into Tai-e [38], a state-of-the-art Java static analysis framework, for evaluation. For each accelerated analysis, the corresponding driver comprises about 100 lines of code. We open-source this driver together with FPGAFlow, providing a concrete reference for integrating FPGAFlow into other static analyzers or compiler infrastructures.

On the FPGA side, to establish a TCP communication channel over Ethernet between the hardware accelerator and the host machine, our system leverages the Processing System (PS)—typically an embedded ARM processor within the FPGA SoC—to execute the lwIP (Lightweight IP) protocol stack. Although our prototype uses lwIP, deploying FPGAFlow on higher-end FPGA platforms can easily support more advanced network stacks or PCIe interfaces to further elevate data transfer bandwidth, as the accelerator architecture is agnostic to the underlying communication medium.

4.4 Potential Extensions

This section discusses potential extensions of FPGAFLOW that are not implemented in the current prototype but could be useful, including integration with LLVM/Clang and adaptation to non-set-based dataflow analyses.

4.4.1 Integration with LLVM/Clang. In the preceding discussion and in our evaluation, FPGAFLOW is integrated with the Java static-analysis framework Tai-e [38]. Beyond this setting, FPGAFLOW could also be integrated into compilers to accelerate their dataflow analyses. Here, we use LLVM [18] as an example to present a high-level integration idea for applying FPGAFLOW to accelerate live variables analysis in its C-family frontend, Clang [24]. Since the host-side workload mainly involves preparing and transmitting the CFG and the gen/kill facts through our network interface, we outline how these inputs could be constructed from Clang’s internal data structures.

Regarding the CFG, Clang’s CFG uses basic blocks as nodes. To compute statement-level IN/OUT facts, the CFG needs to be adapted to use statements as nodes. Specifically, given Clang’s CFG, for each basic block (represented as CFGBlock), one can traverse its constituent statements (CFGStmt)¹ and treat them as nodes in a new CFG. These statement nodes should be connected sequentially within each basic block, with the last statement of the current basic block connected to the first statement of each successor basic block via CFGBlock::succs(). After constructing this statement-level CFG, the restructuring described in Section 4.1.2 can then be applied before the resulting CFG is sent to FPGAFLOW.

Regarding the gen/kill facts, Clang’s design provides mechanisms that can simplify their construction without explicitly handling the many corner cases of C/C++. Clang encapsulates the gen/kill logic in a TransferFunctions class that implements the transfer functions for liveness. When applied to a statement, it generates and kills liveness facts via the Add* and Remove* APIs. Therefore, we can apply the transfer functions to each statement before running the dataflow analysis, and instrument the Add*/Remove* APIs to record the generated and killed facts, from which the per-statement gen/kill sets can be constructed. Finally, the resulting sets should be encoded as the bitvectors required by FPGAFLOW, chunked, and transmitted to the FPGA.

4.4.2 A High-Level Idea for Adapting FPGAFLOW to Non-Set-Based Dataflow Analyses. Although FPGAFLOW is designed for intra-procedural set-based dataflow analyses, we hope it can serve as a reference point for exploring FPGA-accelerated static analysis beyond our current scope. Accordingly, we use constant propagation [1], a widely used non-set-based dataflow analysis, as a representative example to illustrate a high-level idea for adapting the experience and design of FPGAFLOW beyond the set-based setting.

Adapting FPGAFLOW to constant propagation requires two key changes: (1) adopting a suitable representation of dataflow facts, and (2) implementing the merge and transfer functions with the semantics specific to constant propagation.

Dataflow Fact Representation. Unlike set-based dataflow analysis, where the dataflow facts are per-CFG-node sets, constant propagation typically uses a per-CFG-node map from each variable to its abstract value. The abstract value summarizes, at a given program point, whether a variable is a specific constant, not a constant, or undefined. To represent the per-CFG-node map, we can adapt FPGAFLOW to store in BRAM a 2D array indexed by CFG node and variable. (This requires assigning integer IDs to CFG nodes and variables.) Each array element stores the abstract value of a given variable at a given node. To represent an abstract value, assuming the analyzed variables are n -bit numbers, one option is to use $2n$ bits to encode each abstract value, where each 2-bit

¹A CFGBlock may contain elements beyond CFGStmt nodes (e.g., destructors such as CFGAutomaticObjDtor) that can affect liveness and should be handled as needed; we mention statements here for brevity.

field represents an abstract bit. Specifically, 00 denotes undefined, 01 denotes a constant 0 bit, 10 denotes a constant 1 bit, and 11 denotes not a constant. With this encoding, a $2n$ -bit entry provides a bit-level abstract value for the analyzed variable in constant propagation.

Storing these facts in the FPGA faces the same *spatial feasibility* challenge discussed earlier: the memory footprint can grow superlinearly with procedure size, and constant propagation cannot exploit the distributivity of data-fact operators leveraged by set-based dataflow analysis to mitigate this growth. One possible solution is to maintain a single global map from each variable to its abstract value, rather than a per-CFG-node map, so that the memory required to represent dataflow facts grows linearly with the number of variables. This solution can introduce imprecision, which can be alleviated by using single static assignment (SSA) form [7], an intermediate representation (IR) in which each variable is assigned once.

Transfer/Merge Function Implementation. Under SSA, an explicit merge function is unnecessary: its role is subsumed by the transfer function of ϕ nodes. Accordingly, we remove the Merge Unit (Figure 3) and focus on adapting the Transfer Unit to implement these transfer functions.

Because different CFG nodes (i.e., statements) apply different transfer functions according to their semantics in constant propagation, we can support this behavior by storing, in BRAM, a mapping from each node index to an *instruction* that encodes (1) the operator specifying the transfer computation, (2) the indices of the variables used by the node, and (3) the index of the variable defined by the node—akin to a CPU instruction format. Accordingly, the Transfer Unit needs a computation module for each supported operator, similar to a CPU ALU but operating over the abstract-value domain used for constant propagation. During execution, the Transfer Unit fetches the instruction for the current node, decodes it, reads the abstract values of the used variables from the global map, and dispatches them to the operator-specific computation module to produce a new abstract value for the defined variable. The Update Unit then behaves as before: it checks whether the defined variable’s abstract value has changed; if so, it writes the new value back to the global map and enqueues all affected nodes into the worklist. The key adaptation is that the affected nodes are those that use the updated variable; to support this step, we can maintain in BRAM a mapping from each variable to its use sites (nodes).

5 Evaluation

In this section, we investigate the following research questions for evaluation.

RQ1. To what extent can FPGAFlow accelerate dataflow analysis, and how do the optimizations adopted contribute to the overall performance gain?

RQ2. How does the hyperparameter of FPGAFlow affect its space-time trade-off?

Implementation. FPGAFlow is implemented in 2,244 lines of Verilog and deployed on an FPGA board featuring a Xilinx Zynq-7000 SoC (XC7Z020CLG400-2) running at 55 MHz. Table 1 reports the FPGA resource utilization of FPGAFlow. The on-board processing system, which controls the communication between the host system and the FPGA as described in Section 2, comprises 586 lines of C code and runs on a dual-core ARM Cortex-A9 at 766 MHz with 1 GB DDR3 memory.

Table 1. Resource Utilization of FPGAFlow

Resource	Utilization	Available	Rate (%)
LUT	44037	53200	82.78
LUTRAM	58	17400	0.33
FF	18397	106400	17.29
BRAM	115.50	140	82.50
BUFG	1	32	3.13

Although FPGAFlow is applicable to general set-based dataflow analyses, for convenience in this evaluation, we focus on live variables analysis and reaching definitions analysis for Java. This choice is motivated by two considerations: (1) these two analyses are fundamental and widely used in

Table 2. Overview of Benchmarks Used for Evaluation (Star counts retrieved Jan. 2026).

Category	Project	Description	Stars
Foundational Frameworks	LTS JREs* [27]	Standard libraries of LTS Java Runtimes (8, 11, 17).	—
	Android* [9]	Core framework libraries from AOSP.	—
	Kotlin [15]	Statically typed language targeting the JVM.	52.1k+
Distributed Middleware	Elasticsearch [8]	Distributed search and analytics engine.	75.8k+
	Flink [40]	Unified stream and batch processing framework.	25.7k+
Development Tools	IntelliJ [14]	Core platform source code of IntelliJ IDEA.	19.4k+
	Gradle [10]	Multi-language build automation tool.	18.2k+
Business Applications	Metabase [26]	Business intelligence tool for data visualization and analytics.	45.4k+
	Halo [11]	Content management system for modern publishing.	37.7k+
	RuoYi [46]	Open-source enterprise backend management system.	33.5k+
	Eladmin [48]	Enterprise-grade administration system.	21.9k+
	Grasscutter [41]	High-performance game server emulator.	16.7k+
	Zfile [16]	Lightweight online file storage system.	10.6k+

* Part of larger ecosystems (e.g., OpenJDK, AOSP) without a single star count.

various software engineering tasks, and (2) our implementation is currently adapted with Tai-e [38], a state-of-the-art Java static analyzer that supports them. Here, the infrastructure connecting Tai-e and FPGAFLOW (e.g., CFG and data-fact preprocessing, and host-to-board I/O) comprises 974 lines of Java code, while the analysis-specific adapters for live variables analysis and reaching definitions analysis add 100 and 95 lines of Java code, respectively.

Moreover, we note that this evaluation scope is chosen for convenience and does not limit the generality of FPGAFLOW. In particular, supporting other set-based dataflow analyses simply requires the host software to transmit the corresponding data (e.g., gen/kill bitvector) to FPGAFLOW through its flexible interface, after which the FPGA kernel can execute the analysis automatically. Similarly, integrating FPGAFLOW with other static analyzers can be achieved by implementing a lightweight adapter layer, as discussed in Section 4.3.

Benchmarks. To comprehensively evaluate the efficacy of our accelerator, the benchmarks are curated to encompass a wide range of real-world Java applications, including foundational libraries, distributed middleware, development tools, and business applications. Table 2 provides an overview of the benchmarks used in our evaluation, including their categories, descriptions, and GitHub star counts as a proxy for their impact and relevance in the community.

Baselines. To measure the acceleration gain of FPGAFLOW, we select three baselines for comparison: a software baseline for overall performance comparison, and two hardware baselines for ablation studies to evaluate the impact of specific optimizations adopted in FPGAFLOW.

- **Software Baseline (BASE):** FPGAFLOW works as an attached accelerator to a host CPU, thus its acceleration gain is first measured against the standard software implementation running on that host CPU, i.e., Intel Xeon Silver 4114 CPUs @ 2.20–3.00 GHz with 32 GB memory allocated.
- **Hardware Baseline (No-OPT and SPACE-OPT):** To further evaluate the impact of the optimizations adopted in FPGAFLOW, as an ablation study, we construct two hardware baselines that represent different levels of optimization. No-OPT is the straightforward accelerator mentioned in Section 3, which is a direct hardware translation of the standard algorithm without any of the optimizations we proposed. Since No-OPT scales poorly and cannot support most of the procedures in the

benchmark, we do not provide an implementation for it; rather, in the later evaluation, we calculate the theoretical optimal performance for this benchmark. SPACE-OPT is an intermediate version that incorporates the spatial optimizations (Section 4.1), but does not include the temporal optimizations (Section 4.2). It takes 1,724 lines of Verilog code to implement.

Time Measurement. For procedures analyzed on hardware, we measure execution time using a cycle counter that records the number of cycles required to analyze each procedure. We then sum the cycle counts over these procedures and convert the total into time using the FPGA clock frequency. For procedures analyzed in software, we measure execution time as the accumulated wall-clock time spent invoking Tai-e to perform dataflow analysis over these procedures.

Correctness Validation. To validate the correctness of our FPGA-accelerated approach, we compare the dataflow analysis results produced by FPGAFlow against those generated by the standard software execution running on the host CPU. Across all benchmarks, FPGAFlow yields results completely identical to the software, perfectly matching 75,917,003,766 bits across all benchmarks in the two performed analyses. This provides strong evidence that FPGAFlow maintains functional equivalence with the software while achieving considerable acceleration.

5.1 RQ1. On the Performance Gain of FPGAFlow

To answer RQ1, we evaluate the performance efficiency of our FPGA acceleration by conducting a comparison involving BASE, No-OPT, SPACE-OPT, and FPGAFlow, with the results summarized in Table 3. The table does not account for the overhead caused by I/O communication and host-side preparation, since they are orthogonal to our main contribution and not fully optimized in our implementation; that said, we will discuss their influence later in this section. Below we analyze the results and estimate the contribution of different optimizations we adopted.

Overall Speedup (BASE vs. FPGAFlow). In our evaluation setup, the FPGA-based solution runs on a modest FPGA platform (\$130, 28 nm, 55 MHz), whereas the software baseline runs on a server-grade CPU (\$600, 14 nm, 2.20 GHz–3.00 GHz). Despite this hardware setting, comparing FPGAFlow against BASE yields promising overall speedups across both analyses.

- For live variables analysis, FPGAFlow achieves a geometric mean speedup of 15.45 $\times$, with individual benchmark speedups ranging from 4.8 $\times$ to 40.2 $\times$.
- For reaching definitions analysis, FPGAFlow achieves a geometric mean speedup of 12.32 $\times$, with individual benchmark speedups ranging from 4.6 $\times$ to 29.6 $\times$.

These results also indicate that FPGAFlow provides consistent acceleration across the two dataflow analyses, despite their distinct data distributions and computational characteristics.

Contribution of Temporal Optimization (SPACE-OPT vs. FPGAFlow). The temporal optimization, as discussed in Section 4.2, is designed to further enhance the performance of the accelerator beyond SPACE-OPT by exploiting pipelined parallelism. To estimate its contribution to the overall performance gain, we compare SPACE-OPT with FPGAFlow:

- For live variables analysis, the temporal optimization increases the speedup from 1.57 $\times$ to 15.45 $\times$.
- For reaching definitions analysis, the speedup rises from 1.31 $\times$ to 12.32 $\times$.

These results demonstrate that the temporal optimization is the primary contributor to the overall performance gain of FPGAFlow, as it significantly enhances the hardware efficiency and enables the accelerator to fully leverage its parallel processing capabilities, leading to substantial speedups over the software baseline.

Table 3. Performance comparison of BASE, No-OPT, SPACE-OPT, and FPGAFlow for live variables analysis and reaching definitions analysis (time in seconds). #Proc reports the number of all procedures in each benchmark; reported time in the table is the summation of the execution time of each procedure. BASE reports the execution time of the software baseline. No-OPT reports the estimated theoretically optimal speedup of the straightforward implementation. For SPACE-OPT and FPGAFlow, TOTAL denotes the overall execution time, including the hardware execution time for FPGA-analyzable procedures and the software analysis time for procedures that cannot be analyzed on the FPGA. SPD. denotes the speedup over BASE. The GEOMEAN reports the geometric mean of the speedups across all benchmarks; it is a more appropriate and robust (although numerically smaller) metric than the arithmetic mean.

Benchmark	#Proc	Live Variables						Reaching Definitions					
		BASE	No-OPT	SPACE-OPT		FPGAFlow		BASE	No-OPT	SPACE-OPT		FPGAFlow	
				TOTAL	SPD.	TOTAL	SPD.			TOTAL	SPD.	TOTAL	SPD.
jdk8	162255	10.483	1.20×	6.244	1.7×	0.521	20.1×	10.990	1.19×	6.754	1.6×	0.585	18.8×
jdk11	222221	24.499	0.73×	62.51	0.4×	4.530	5.4×	21.490	0.70×	63.16	0.3×	4.616	4.7×
jdk17	202088	22.557	0.74×	65.26	0.3×	4.714	4.8×	21.878	0.71×	65.83	0.3×	4.787	4.6×
grasscutter	827890	20.999	2.31×	3.969	5.3×	0.522	40.2×	18.049	2.08×	4.742	3.8×	0.610	29.6×
eladmin	704229	21.576	1.49×	9.944	2.2×	1.214	17.8×	19.408	1.32×	11.34	1.7×	1.568	12.4×
halo	381069	16.349	1.22×	11.17	1.5×	1.508	10.8×	15.223	1.13×	11.96	1.3×	2.121	7.2×
android	619937	20.526	1.34×	11.52	1.8×	1.016	20.2×	19.284	1.24×	12.80	1.5×	1.146	16.8×
ruoyi	602444	24.371	1.42×	11.65	2.1×	1.797	13.6×	21.853	1.26×	13.77	1.6×	2.070	10.6×
metabase	612811	37.953	1.11×	28.03	1.4×	2.460	15.4×	34.071	1.05×	30.22	1.1×	2.736	12.5×
kotlin	278523	12.214	1.53×	4.839	2.5×	0.452	27.0×	10.091	1.46×	4.586	2.2×	0.468	21.6×
gradle	611615	25.546	1.31×	14.18	1.8×	1.339	19.1×	22.315	1.23×	14.53	1.5×	1.467	15.2×
elastic	631287	28.512	1.09×	23.76	1.2×	2.038	14.0×	26.878	1.03×	25.27	1.1×	2.239	12.0×
flink	465916	17.090	1.65×	5.982	2.9×	0.697	24.5×	13.861	1.44×	6.717	2.1×	0.762	18.2×
idea	929897	42.161	1.23×	27.94	1.5×	2.605	16.2×	36.701	1.13×	28.99	1.3×	2.801	13.1×
zfile	288919	13.828	1.32×	7.070	2.0×	0.970	14.3×	14.140	1.27×	8.067	1.8×	1.326	10.7×
GEOMEAN	–	–	1.26×	–	1.57×	–	15.45×	–	1.18×	–	1.31×	–	12.32×

Contribution of Spatial Optimization (No-OPT vs. SPACE-OPT). The spatial optimization, as discussed in Section 4.1, is designed to scale up the accelerator to support larger procedures under the limited on-chip resources. Although the optimization is not designed for performance speedup, here, we still compare the straightforward accelerator (No-OPT, described in Section 3) with the spatially optimized version (SPACE-OPT) to estimate its contribution to the overall performance gain.

- For live variables analysis, the spatial optimization increases the speedup from 1.26×
- For reaching definitions analysis, the speedup rises from 1.18×

Notably, we did not implement No-OPT on the FPGA as it is not expected to yield meaningful results; instead, we provide an analytic estimation of the upper bound of its speedup: (1) we filter out the procedures that exceed the accelerator’s memory budget, as they would fall back to software execution and thus yield no speedup; (2) for the remaining procedures, we assume No-OPT can process as fast as SPACE-OPT can. Under this assumption, the speedup of No-OPT is barely 1.26×

and 1.18×

because it fails to support a significant portion of procedures due to memory constraints, and thus cannot leverage the FPGA’s parallel processing capabilities for those cases.

In contrast, SPACE-OPT successfully supports over 99.99% of procedures by adopting the spatial optimizations, and thus can fully utilize the FPGA for acceleration. Concretely, under the current BRAM budget of our FPGA board (4.8 Mbits), the largest CFG that SPACE-OPT can analyze contains

16,384 nodes. (We later show how to derive this CFG-size limit in Theorem 2.) The remaining 0.01% of procedures (0–5 procedures per benchmark) exceed this limit and thus fall back to software execution, i.e., they are analyzed on the host CPU by Tai-e when they do not fit on the FPGA.

Discussions on Less-Accelerated Cases. We also observed that jdk11 and jdk17 show lower speedups compared to other benchmarks. We attribute this primarily to the presence of certain large and complex procedures within these libraries (e.g., with CFG size $> 2^{12}$ nodes). Such large procedures typically induce larger dataflow fact domains (e.g., bitvectors with over 1,000 bits); as described in Section 4.1.1, we split these large bitvectors into substantial numbers of smaller chunks to fit within the FPGA’s memory budget, which significantly increases the number of iterations required for processing and thus leads to lower speedups.

Notably, we observed that such cases are relatively rare in real-world software projects, and are particularly prevalent in the JDK libraries, probably due to specific architectural design requirements. This observation indicates that FPGAFlow can still provide significant acceleration for the vast majority of procedures in real-world software, though it may be less but still beneficial for exceptionally large procedures.

Discussions on I/O and host-side preparation. The FPGA execution times reported above exclude (1) I/O communication overhead between the host CPU and the FPGA and (2) host-side preparation overhead (CFG restructuring, serialization, GEN/KILL bitvector construction, and chunking). This is because communication optimization and software preprocessing are largely orthogonal to the main contribution of this work, namely the accelerator design for dataflow analysis. In the following paragraphs, we discuss their potential impact on our experimental results.

To facilitate rapid validation, our current prototype uses Ethernet with the AXI-Lite protocol for I/O. This choice significantly reduces engineering effort compared to high-speed I/O solutions, but it can unduly degrade the speedup. Because efficient FPGA communication is well studied and the platform can, in principle, approach its theoretical bandwidth with a carefully engineered design (e.g., AXI-Full and PCIe), we provide a theoretical estimate of I/O overhead—and its impact on overall performance—under an optimized communication setting that can approach the theoretical bandwidth of the platform. According to Xilinx documentation [13], the FPGA platform we use can provide up to 4.8 GB/s bandwidth with a more advanced implementation. Accordingly, we estimate I/O time by computing the total data-transfer volume and dividing it by 4.8 GB/s. The **I/O Overhead and Spd. (+I/O)** columns in Table 4 report the resulting impact for reaching definitions analysis. On average, accounting for I/O reduces the speedup of FPGAFlow from 12.3× to 11.0×. For live variables analysis (not shown in the table for brevity), the speedup decreases from 15.4× to 13.5×. Overall, under an optimized communication subsystem, I/O overhead has only a modest impact on the achieved speedups.

For host-side preparation overhead, the **Preparation Overhead and Spd. (+Prep)** columns in Table 4 report the resulting impact for reaching definitions analysis. Our measurements show that CFG restructuring and serialization overheads are negligible, as expected, because only a small number of nodes require restructuring (Section 4.1.2). In contrast, constructing and chunking the GEN/KILL bitvectors incurs a relatively high overhead, which we did not anticipate. We attribute this overhead primarily to the lack of optimization in our current implementation and expect that further engineering can reduce it. For example, this GEN/KILL bitvector preparation can be pipelined with FPGA acceleration: after preparing the GEN/KILL bitvectors for one procedure, the host immediately sends them to the FPGA while preparing the GEN/KILL bitvectors for the next procedure. After accounting for host-side preparation overhead, FPGAFlow achieves an average speedup of 6.98× for reaching definitions analysis and 5.21× for live variables analysis (not shown in the table for brevity) over the baseline. When accounting for both I/O and host-side preparation

Table 4. I/O and host-side preparation overhead of reaching definitions analysis. I/O Nodes and Vol. report the number of transferred nodes (in millions) and the data-transfer volume (in GB), respectively. I/O Time is estimated assuming a bandwidth of 4.8 GB/s. Preparation times are measured from the current implementation and include a CFG part (CFG restructuring and serialization) and a BV part (constructing and chunking the GEN/KILL bitvectors). Spd. (Orig.) excludes all overheads; Spd. (+I/O) includes I/O time; Spd. (+Prep) includes preparation time; Spd. (+Both) includes both I/O and preparation.

Benchmark	I/O Overhead			Preparation Overhead			Spd.	Spd.	Spd.	Spd.
	#Nodes	Vol. (GB)	Time (s)	CFG (s)	BV (s)	Total (s)	(Orig.)	(+I/O)	(+Prep)	(+Both)
jdk8	23.4M	0.435	0.091	0.09	0.52	0.61	18.8×	16.3×	9.19×	8.55×
jdk11	243.5M	4.536	0.945	0.14	1.40	1.54	4.7×	3.9×	3.49×	3.03×
jdk17	254.6M	4.742	0.988	0.15	1.47	1.62	4.6×	3.8×	3.42×	2.96×
grasscutter	14.5M	0.269	0.056	0.17	1.02	1.19	29.6×	27.1×	10.0×	9.72×
eladmin	34.4M	0.641	0.134	0.26	1.01	1.27	12.4×	11.4×	6.85×	6.53×
halo	38.7M	0.722	0.150	0.14	0.83	0.97	7.2×	6.7×	4.92×	4.70×
android	42.4M	0.790	0.165	0.16	0.86	1.02	16.8×	14.7×	8.90×	8.27×
ruoyi	39.3M	0.732	0.152	0.16	1.06	1.22	10.6×	9.8×	6.64×	6.35×
metabase	104.6M	1.949	0.406	0.22	1.57	1.79	12.5×	10.8×	7.53×	6.91×
kotlin	14.6M	0.272	0.057	0.09	0.52	0.61	21.6×	19.2×	9.37×	8.89×
gradle	48.0M	0.893	0.186	0.21	1.17	1.38	15.2×	13.5×	7.84×	7.36×
elastic	89.3M	1.664	0.347	0.20	1.15	1.35	12.0×	10.4×	7.49×	6.83×
flink	21.4M	0.399	0.083	0.13	0.63	0.76	18.2×	16.4×	9.11×	8.64×
idea	95.3M	1.775	0.370	0.33	1.54	1.87	13.1×	11.6×	7.86×	7.28×
zfile	24.2M	0.451	0.094	0.11	0.56	0.67	10.7×	10.0×	7.08×	6.77×
GEOMEAN	–	–	–	–	–	–	12.3×	11.0×	6.98×	6.50×

overheads, the corresponding speedups are 6.50× and 4.86×. Although these numbers are lower than the originally reported 12.32× and 15.45×, respectively, they still show the feasibility and potential of FPGA acceleration, which is the focus of this paper.

5.2 RQ2. On the Space-time Trade-off in FPGAFlow

The space-time trade-off is a fundamental consideration in the design of FPGA accelerators, as the on-chip resources are strictly limited and must be carefully allocated to achieve optimal performance.

In FPGAFlow, we adopt a key hyperparameter, i.e., the width of data facts (W_{fact}), tunable to balance the trade-off between space scalability and runtime performance. Specifically, increasing the width of data facts enables the accelerator to process more data facts in parallel in each iteration, thereby improving runtime performance. However, it also increases the per-procedure memory requirement, causing more procedures to exceed the FPGA's memory budget and fall back to software execution, which in turn increases the overall execution time.

Below, we first provide a theoretical analysis guiding the selection of the data fact width, and then report the empirical results of choosing different data fact widths for FPGAFlow.

Theoretical Analysis. To estimate how the width of data facts affects the space-time trade-off of FPGAFlow, we want to answer the question: *Given a certain data fact width, what is the maximum procedure that can be supported by FPGAFlow under the on-chip memory constraints?*

Formally, we want to derive the relationship between the following factors:

- The on-chip memory budget (M): the total amount of on-chip memory available for storing the procedure's CFG structure and other facilities used in `FPGAFlow`.
- The width of data facts (W_{fact}): the tunable hyperparameter that determines the number of data facts processed in parallel in each iteration.
- The procedure size (N): the number of CFG nodes in the procedure to be analyzed.

Theorem 2 (Bound of the width of data facts). *Given the parameters defined above, the width of data facts W_{fact} must satisfy the following inequality to ensure that `FPGAFlow` can support a procedure of size N under the on-chip memory budget M :*

$$W_{fact} \leq \frac{1}{5} \left(\frac{M - C}{N} - 6 \log_2 N \right)$$

where C is a small constant representing the overhead from the FIFO and BRAM generator logic.

PROOF. By definition, we have the following relationship:

$$M \geq \text{Memory required for Procedure CFG} + \text{Memory required for FPGA facilities}$$

- For the first part, the memory required for storing the procedure's CFG structure is given by the memory required for Gen/Kill sets and three dataflow fact sets, each taking $W_{fact} \times N + C_1$ bits, where C_1 is a small constant representing the overhead from the BRAM generator logic.
- For the second part, the memory required for FPGA facilities is given by the memory required for the FIFO queue, Predecessor/Successor sets. These data structures store node indices, with each requiring $2N \log_2 N + C_2$ bits. Here, 2 is the maximum number of predecessors/successors per node after CFG restructuring; $\log_2 N$ is the bit width required to index N nodes; C_2 is a small constant representing the overhead from the FIFO and BRAM generator logic.

Thus, we have the following inequality:

$$M \geq 5 \cdot W_{fact} \cdot N + 6 \cdot N \log_2 N + C,$$

where C is a small constant derived from C_1 and C_2 . Finally, rearranging this inequality gives us the bound on W_{fact} as stated in the theorem. $\square$

Specifically, on the FPGA platform we adopted with $M = 4.8 \times 10^6$ bits, targeting canonical procedures with $N = 20,000$ CFG nodes, the resulting W_{fact} would be approximately 30.

Practical Considerations. Theorem 2 provides a theoretical upper bound when selecting the data fact width for `FPGAFlow`. However, slightly exceeding this bound (specifically, 30) is still acceptable due to the following practical considerations:

- (1) Choosing a W_{fact} in the form of 2^k (e.g., 32-bit, 64-bit, 128-bit) often leads to better hardware efficiency due to the architectural design of common FPGA platforms, even though it may slightly exceed the theoretical bound;
- (2) When not exceeding the bound too much, a substantial number of procedures can still be supported on the FPGA, as the distribution of procedure sizes is often skewed with a long tail (i.e., most procedures are small, and only a few are large); and
- (3) For the small number of procedures that exceed the bound and thus cannot be supported by the accelerator, we can fall back to software execution for them, which would not significantly affect the overall performance given their rarity.

To further illustrate these practical considerations, Table 5 presents the empirical results for `FPGAFlow` under 32-bit and 64-bit data fact widths (i.e., the practical widths slightly exceeding the theoretical bound 30) across the evaluated benchmarks. In this table, the column HW denotes the hardware execution time for procedures that fit within the FPGA's memory budget, the column

Table 5. Hardware execution time, software fallback time, and resulting speedup under 32-bit and 64-bit data fact widths for live variables analysis and reaching definitions analysis across the evaluated benchmarks. Here, HW denotes the hardware execution time, SW denotes the execution time of procedures that fall back to software, and SPD. denotes the speedup over the software baseline (BASE).

Benchmark	Live Variables						Reaching Definitions					
	$W_{fact} = 32$			$W_{fact} = 64$			$W_{fact} = 32$			$W_{fact} = 64$		
	HW	SW	SPD.	HW	SW	SPD.	HW	SW	SPD.	HW	SW	SPD.
jdk8	0.521	0.000	20.1×	0.265	0.000	39.6×	0.585	0.000	18.8×	0.300	0.000	36.6×
jdk11	4.530	0.000	5.4×	1.724	1.102	8.7×	4.616	0.000	4.7×	1.770	1.096	7.5×
jdk17	4.714	0.000	4.8×	1.781	1.060	7.9×	4.787	0.000	4.6×	1.819	1.094	7.5×
grasscutter	0.522	0.000	40.2×	0.357	0.000	58.8×	0.610	0.000	29.6×	0.405	0.000	44.6×
eladmin	0.912	0.302	17.8×	0.467	0.518	21.9×	1.044	0.524	12.4×	0.521	0.718	15.7×
halo	0.887	0.621	10.8×	0.289	1.259	10.6×	0.964	1.157	7.2×	0.322	1.933	6.8×
android	1.016	0.000	20.2×	0.403	0.739	18.0×	1.146	0.000	16.8×	0.467	0.675	16.9×
ruoyi	0.993	0.804	13.6×	0.504	1.097	15.2×	1.174	0.896	10.6×	0.586	1.158	12.5×
metabase	2.232	0.228	15.4×	0.963	0.873	20.7×	2.498	0.238	12.5×	1.089	0.818	17.9×
kotlin	0.452	0.000	27.0×	0.241	0.098	36.0×	0.468	0.000	21.6×	0.235	0.091	31.0×
gradle	1.237	0.102	19.1×	0.644	0.255	28.4×	1.333	0.134	15.2×	0.675	0.268	23.7×
elastic	1.924	0.114	14.0×	0.614	1.426	14.0×	2.131	0.108	12.0×	0.721	1.666	11.3×
flink	0.577	0.120	24.5×	0.303	0.256	30.6×	0.665	0.097	18.2×	0.351	0.222	24.2×
idea	2.352	0.253	16.2×	1.038	1.096	19.8×	2.581	0.220	13.1×	1.115	1.358	14.8×
zfile	0.597	0.373	14.3×	0.298	0.484	17.7×	0.690	0.636	10.7×	0.347	0.749	12.9×
GEOMEAN	–	–	15.45×	–	–	19.97×	–	–	12.32×	–	–	16.21×

SW denotes the execution time of procedures that fall back to software on the host CPU, and the column SPD. denotes the resulting speedup over the software baseline (BASE), accounting for both hardware execution and software fallback.

As shown in Table 5, when moving from a 32-bit to a 64-bit data fact width, the execution time in SW increases, because the wider data fact width occupies more memory budget, causing more procedures to exceed the FPGA's memory limit and fall back to software execution, as observed in halo (the speedup regresses from 10.8×

As a result, the overall speedup of the 64-bit configuration is marginally higher than that of the 32-bit configuration, increasing from 15.45×

Takeaway Message. The data fact width is a critical hyperparameter that directly affects the space-time trade-off of FPGAFlow. When there are plenty of on-chip resources, selecting a wider

data fact width within the theoretical bound (Theorem 2) can yield better performance by enabling more parallel processing of data facts. However, once this bound is exceeded, as the data fact width increases, the speedup could be marginal due to more frequent software fallback. Therefore, it is crucial to carefully select the data fact width to balance the trade-off between space scalability and runtime performance, guided by the theoretical analysis and empirical results presented above.

6 Related Work

This work focuses on dataflow analysis acceleration from a hardware perspective, utilizing FPGA platforms. While there is no directly comparable hardware acceleration work, in this section, we discuss related work that addresses the problem from a software perspective, leveraging various general-purpose computing platforms, such as multi-core CPUs, GPUs, and distributed systems.

These existing software approaches are not directly applicable to FPGA acceleration, since FPGA acceleration emphasizes managing limited on-chip resources, whereas CPUs, distributed systems, and GPUs face other challenges: CPU-based approaches focus on shared-memory multi-core parallelization, distributed systems emphasize scaling out via partitioning and communication, and GPU-based approaches target throughput-oriented SIMD/SIMT parallelism and memory bandwidth utilization.

In the following, we discuss dataflow analysis acceleration work on these three platforms, and then briefly discuss FPGA accelerators for other algorithms (e.g., deep learning) for context, as they are not directly applicable to dataflow analysis.

CPU Parallelization. Our work focuses on intra-procedural dataflow analysis as a first step toward FPGA acceleration; in contrast, due to their general-purpose programmability, CPU-based acceleration has been explored across a broader range of program analyses, including inter-procedural, context-sensitive, and IFDS analyses. On this platform, the main technical concern is typically multi-core scheduling over shared memory: coordinating many threads while controlling synchronization overhead and contention, e.g., via lock-free data structures, careful worklist scheduling, or relaxed dependencies to enable pipelined execution. Although these challenges differ from our FPGA setting (where the bottleneck is limited on-chip resources rather than scheduling), this line of work still offers some inspiration, e.g., pipelined workflows and dependency-management strategies.

For example, Rodriguez and Lhoták [31] parallelize IFDS analyses by expressing the analysis in an actor model, where message passing reduces shared-memory contention compared to thread-centric designs. Shi and Zhang [33] improve parallelism via pipelining in bottom-up analysis, relaxing strict calling dependencies so multiple analysis stages can overlap across cores. Complementary to in-memory multi-core scheduling, Wang et al. [42] (Graspan) reformulate interprocedural analysis as large-scale graph reachability and exploit disk-based, out-of-core processing to scale beyond main memory on a single machine. Similarly, Zuo et al. [50] (Chianina) decouple analysis logic from scalability by modeling flow-sensitivity with an evolving graph and using disk-based partitioning to scale context-/flow-sensitive analyses on commodity CPUs.

Distributed Systems. Our work focuses on acceleration achieved by attaching a single FPGA board to a single machine; in contrast, distributed implementations are largely motivated by *very large codebases* that exceed the capacity of a single machine, and thus target scale-out execution across a cluster. Typical techniques therefore emphasize scaling out via partitioning and communication: assigning tasks across workers, exchanging summaries or propagation messages, and coordinating convergence with communication-aware scheduling and load balancing. Because these designs fundamentally assume multi-node memory and network communication, they are less directly applicable to our FPGA setting.

Representative work includes BigSpa [49], which formulates interprocedural static analysis as distributed graph reachability and scales it out in the cloud. Its design centers on partitioning massive program graphs and reducing communication overhead, e.g., via fine-grained partitioning for load balancing and communication-reduction optimizations to speed up convergence. Building on this direction, BigDataflow [37] reinterprets classic worklist-based dataflow analysis in a vertex-centric distributed graph-processing model and provides a general interprocedural framework atop a big-data ecosystem.

GPU-Based Implementation. While an FPGA can be customized to a specific (potentially irregular) analysis algorithm, a GPU requires the algorithm to conform to its fixed SIMT execution model—irregular logic often triggers uncoalesced memory accesses, workload imbalance, and branch divergence. Accordingly, GPU-based approaches typically adjust irregular logic to better match throughput-oriented SIMD/SIMT parallelism and maximize memory bandwidth utilization, e.g., by reducing memory operations and using matrix-based data structures.

Representative examples include EigenCFA [28] that targets higher-order control-flow analysis, reducing branches by reinterpreting analysis using linear algebra operations to fit the GPU architecture. Blaß and Philippsen [4] parallelize interprocedural predicate-propagation fixpoint computation on GPUs using a largely synchronization-free design with checksum-based detection/repair for SIMT-level races, and GDroid [45], which accelerates IDFG construction for Android by alleviating GPU underutilization (e.g., dynamic allocations, divergence, and imbalance) via matrix-style data layouts and worklist merging.

Other Related Work. Though FPGA acceleration for dataflow analysis is unexplored, FPGAs are utilized in other domains, i.e., dense numeric kernels (deep learning) or streaming service pipelines (cloud). They are less related to our setting, but still offer useful optimization insights. In deep learning, prior work emphasizes hardware-software co-design, using techniques such as analytical design-space exploration, quantization, and specialized schedulers to improve data reuse and handle sparsity/irregular accesses [12, 30, 47]. In cloud computing and datacenter, prior work focuses on integrating multiple FPGAs at scale to provide low-latency, high-throughput acceleration, focusing on communication between FPGAs rather than single-board design [5, 29].

7 Conclusion

Designing specialized hardware to accelerate static analysis remains unexplored in the research community. As the first exploration in this direction, this work introduces FPGAFlow, a specialized FPGA-based hardware design for accelerating intra-procedural set-based dataflow analysis. FPGAFlow addresses the spatial infeasibility challenge of analyzing large procedures on FPGAs and further exploits hardware-specific parallelism to improve performance. Our evaluation shows that even on a modest FPGA platform, FPGAFlow achieves promising speedups on two representative dataflow analyses—averaging 15.45× for live variables and 12.32× for reaching definitions—compared to a state-of-the-art software analyzer running on a high-end CPU. We also explore how to configure a hyperparameter to achieve ideal performance when deploying FPGAFlow on FPGAs with different on-chip memory budgets. We hope our exploration in this work (including the open-sourcing of FPGAFlow) could offer helpful insights for future research on hardware-accelerated static analysis.

Data-Availability Statement

We provide an artifact [43], available at <https://doi.org/10.5281/zenodo.19045151>, to facilitate the reuse of FPGAFlow and the reproduction of its experimental results. The artifact contains the full open-source implementation of FPGAFlow, including the FPGA hardware design and host-side software for driving and controlling the FPGA accelerator. It also provides a tutorial for deploying

FPGAFlow on an FPGA board and documentation for reproducing the experiments. In addition, because the FPGA platform used in our evaluation is typically not accessible to readers, the artifact includes screen recordings of the experiments as evidence of successful reproduction.

Acknowledgments

We sincerely thank the anonymous reviewers for their thoughtful and constructive comments. We are also grateful to Hailong Zhu for valuable advice on the design of FPGAFlow. We thank Yufei Liang and Jinpeng Wang for their helpful suggestions on improving the paper. This work was supported in part by National Key R&D Program of China (Grant No. 2023YFB4503804), National Natural Science Foundation of China (Grant Nos. 62402210 and 62272217), Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (Grant No. JYB2025XDXM118), and 111 Center (Grant No. B26023). We thank the Collaborative Innovation Center of Novel Software Technology and Industrialization, Jiangsu, China, for its support. Tian Tan, the co-corresponding author, was also supported by the Xiaomi Foundation.

References

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. 2006. *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Addison-Wesley Longman Publishing Co., Inc., USA. <https://dl.acm.org/doi/10.5555/1177220>
- [2] Frances E. Allen. 1970. Control flow analysis. *SIGPLAN Notices* 5, 7 (July 1970), 1–19. doi:10.1145/390013.808479
- [3] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. Flowdroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps. *ACM sigplan notices* 49, 6 (2014), 259–269. doi:10.1145/2666356.2594299
- [4] Thorsten Blaß and Michael Philippsen. 2019. GPU-accelerated fixpoint algorithms for faster compiler analyses. In *Proceedings of the 28th International Conference on Compiler Construction*. 122–134. doi:10.1145/3302516.3307352
- [5] Adrian M Caulfield, Eric S Chung, Andrew Putnam, Hari Angepat, Jeremy Fowers, Michael Haselman, Stephen Heil, Matt Humphrey, Puneet Kaur, Joo-Young Kim, et al. 2016. A cloud-scale acceleration architecture. In *2016 49th Annual IEEE/ACM international symposium on microarchitecture (MICRO)*. IEEE, 1–13. doi:10.1109/MICRO.2016.7783710
- [6] Menglong Chen, Tian Tan, Minxue Pan, and Yue Li. 2025. PacDroid: A Pointer-Analysis-Centric Framework for Security Vulnerabilities in Android Apps. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (Ottawa, Ontario, Canada) (ICSE '25)*. IEEE Press, 2803–2815. doi:10.1109/ICSE55347.2025.00208
- [7] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck. 1989. An efficient method of computing static single assignment form. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (Austin, Texas, USA) (POPL '89)*. Association for Computing Machinery, New York, NY, USA, 25–35. doi:10.1145/75277.75280
- [8] Elasticsearch B.V. 2026. Elasticsearch: Free and Open, Distributed, RESTful Search Engine. <https://github.com/elastic/elasticsearch>. Accessed: Jan. 2026.
- [9] Google LLC. 2026. Android SDK Platform. <https://developer.android.com/studio>. Accessed: Jan. 2026.
- [10] Gradle, Inc. 2026. Gradle Build Tool. <https://github.com/gradle/gradle>. Accessed: Jan. 2026.
- [11] Halo Team. 2026. Halo: A Powerful and Easy-to-Use Content Management System. <https://github.com/halo-dev/halo>. Accessed: Jan. 2026.
- [12] Song Han, Junlong Kang, Huizi Mao, Yiming Hu, Xin Li, Yubin Li, Dongliang Xie, Hong Luo, Song Yao, Yu Wang, et al. 2017. ESE: Efficient speech recognition engine with sparse lstm on fpga. In *Proceedings of the 2017 ACM/SIGDA international symposium on field-programmable gate arrays*. 75–84. doi:10.1145/3020078.3021745
- [13] Xilinx Inc. 2014. *Zynq-7000 All programmable SoC technical reference manual (UG585)*. Technical Report. Tech. rep., Xilinx, 2014, <https://www.xilinx.com/support/documentation>
- [14] JetBrains s.r.o. 2026. IntelliJ IDEA Community Edition Source Code. <https://github.com/JetBrains/intellij-community>. Accessed: Jan. 2026.
- [15] JetBrains s.r.o. 2026. Kotlin: The Kotlin Programming Language. <https://github.com/JetBrains/kotlin>. Accessed: Jan. 2026.
- [16] Jun Zhao. 2026. ZFile: Online File Storage and Management System. <https://github.com/zfile-dev/zfile>. Accessed: Jan. 2026.
- [17] Ken Kennedy. 1972. A global flow analysis algorithm. *International Journal of Computer Mathematics* 3, 1-4 (1972), 5–15. doi:10.1080/00207167108803048

- [18] Chris Lattner and Vikram Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization* (Palo Alto, California) (CGO '04). IEEE Computer Society, USA, 75. doi:10.1109/CGO.2004.1281665
- [19] Sungho Lee, Julian Dolby, and Sukyoung Ryu. 2016. HybriDroid: static analysis framework for Android hybrid applications. In *Proceedings of the 31st IEEE/ACM international conference on automated software engineering*. 250–261. doi:10.1145/2970276.2970368
- [20] Yue Li, Tian Tan, Anders Möller, and Yannis Smaragdakis. 2018. Precision-guided context sensitivity for pointer analysis. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 141 (Oct. 2018), 29 pages. doi:10.1145/3276511
- [21] Yue Li, Tian Tan, Anders Möller, and Yannis Smaragdakis. 2020. A Principled Approach to Selective Context Sensitivity for Pointer Analysis. *ACM Trans. Program. Lang. Syst.* 42, 2, Article 10 (May 2020), 40 pages. doi:10.1145/3381915
- [22] Yue Li, Tian Tan, Yifei Zhang, and Jingling Xue. 2016. Program Tailoring: Slicing by Sequential Criteria. In *30th European Conference on Object-Oriented Programming (ECOOP 2016)* (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 56), Shriram Krishnamurthi and Benjamin S. Lerner (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 15:1–15:27. doi:10.4230/LIPIcs.ECOOP.2016.15
- [23] Yufei Liang, Teng Zhang, Ganlin Li, Tian Tan, Chang Xu, Chun Cao, Xiaoxing Ma, and Yue Li. 2025. Pointer Analysis for Database-Backed Applications. *Proc. ACM Program. Lang.* 9, PLDI, Article 204 (June 2025), 25 pages. doi:10.1145/3729307
- [24] LLVM Developer Group. 2026. Clang: A C language family frontend for LLVM. <https://clang.llvm.org/>.
- [25] Wenjie Ma, Shengyuan Yang, Tian Tan, Xiaoxing Ma, Chang Xu, and Yue Li. 2023. Context Sensitivity without Contexts: A Cut-Shortcut Approach to Fast and Precise Pointer Analysis. *Proc. ACM Program. Lang.* 7, PLDI, Article 128 (June 2023), 26 pages. doi:10.1145/3591242
- [26] Metabase, Inc. 2026. Metabase: The Easiest, Fastest Way to Get Business Intelligence. <https://github.com/metabase/metabase>. Accessed: Jan. 2026.
- [27] Oracle Corporation and/or its affiliates. 2026. OpenJDK: Open Source Implementation of the Java Platform, Standard Edition. <https://openjdk.org/>. Accessed: Jan. 2026.
- [28] Tarun Prabhu, Shreyas Ramalingam, Matthew Might, and Mary Hall. 2011. EigenCFA: Accelerating Flow Analysis with GPUs. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (New York, NY, USA, 2011-01-26) (POPL '11). Association for Computing Machinery, 511–522. doi:10.1145/1926385.1926445
- [29] Andrew Putnam, Adrian M. Caulfield, Eric S. Chung, Derek Chiou, Kypros Constantinides, John Demme, Hadi Esmaeilzadeh, Jeremy Fowers, Gopi Prashanth Gopal, Jan Gray, Michael Haselman, Scott Hauck, Stephen Heil, Amir Hormati, Joo-Young Kim, Sitaram Lanka, James Larus, Eric Peterson, Simon Pope, Aaron Smith, Jason Thong, Phillip Yi Xiao, and Doug Burger. 2015. A Reconfigurable Fabric for Accelerating Large-Scale Datacenter Services. *IEEE Micro* 35, 3 (May 2015), 10–22. doi:10.1109/MM.2015.42
- [30] Jiantao Qiu, Jie Wang, Song Yao, Kaiyuan Guo, Boxun Li, Erjin Zhou, Jincheng Yu, Tianqi Tang, Ningyi Xu, Sen Song, et al. 2016. Going deeper with embedded FPGA platform for convolutional neural network. In *Proceedings of the 2016 ACM/SIGDA international symposium on field-programmable gate arrays*. 26–35. doi:10.1145/2847263.2847265
- [31] Jonathan Rodriguez and Ondřej Lhoták. 2011. Actor-based parallel dataflow analysis. In *Compiler Construction: 20th International Conference, CC 2011, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2011, Saarbrücken, Germany, March 26–April 3, 2011. Proceedings 20*. Springer, 179–197. doi:10.1007/978-3-642-19861-8_11
- [32] Caitlin Sadowski, Edward Aftandilian, Alex Eagle, Liam Miller-Cushon, and Ciera Jaspán. 2018. Lessons from building static analysis tools at google. *Commun. ACM* 61, 4 (2018), 58–66. doi:10.1145/3188720
- [33] Qingkai Shi and Charles Zhang. 2020. Pipelining bottom-up data flow analysis. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 835–847. doi:10.1145/3377811.3380425
- [34] Ting Su, Ke Wu, Weikai Miao, Geguang Pu, Jifeng He, Yuting Chen, and Zhendong Su. 2017. A survey on data-flow testing. *ACM computing surveys (CSUR)* 50, 1 (2017), 1–35. doi:10.1145/3020266
- [35] Yulei Sui and Jingling Xue. 2016. SVF: interprocedural static value-flow analysis in LLVM. In *Proceedings of the 25th international conference on compiler construction*. 265–266. doi:10.1145/2892208.2892235
- [36] Yulei Sui and Jingling Xue. 2020. Value-Flow-Based Demand-Driven Pointer Analysis for C and C++. *IEEE Transactions on Software Engineering* 46, 8 (2020), 812–835. doi:10.1109/TSE.2018.2869336
- [37] Zewen Sun, Duanchen Xu, Yiyu Zhang, Yun Qi, Yueyang Wang, Zhiqiang Zuo, Zhaokang Wang, Yue Li, Xuandong Li, Qingda Lu, et al. 2023. BigDataflow: A distributed interprocedural dataflow analysis framework. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1431–1443. doi:10.1145/3611643.3616348
- [38] Tian Tan and Yue Li. 2023. Tai-e: A developer-friendly static analysis framework for java by harnessing the good designs of classics. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1093–1105. doi:10.1145/3597926.3598120

- [39] Tian Tan, Yue Li, Xiaoxing Ma, Chang Xu, and Yannis Smaragdakis. 2021. Making pointer analysis more precise by unleashing the power of selective context sensitivity. *Proc. ACM Program. Lang.* 5, OOPSLA, Article 147 (Oct. 2021), 27 pages. doi:10.1145/3485524
- [40] The Apache Software Foundation. 2026. Apache Flink: Stateful Computations over Data Streams. <https://github.com/apache/flink>. Accessed: Jan. 2026.
- [41] The Grasscutters Team. 2025. Grasscutter: A Server Software Reimplementation for a Certain Anime Game. <https://github.com/Grasscutters/Grasscutter>. Accessed: Jan. 2026.
- [42] Kai Wang, Aftab Hussain, Zhiqiang Zuo, Guoqing Xu, and Ardalan Amiri Sani. 2017. Graspan: A Single-machine Disk-based Graph System for Interprocedural Static Analyses of Large-scale Systems Code. *SIGPLAN Not.* 52, 4 (April 2017), 389–404. doi:10.1145/3093336.3037744
- [43] Fang Wei, Qinlin Chen, Nairen Zhang, Jiakai Cui, Tian Tan, Zhiqiang Zuo, and Yue Li. 2026. *When FPGA Meets Dataflow Analysis: An Explorative Step (Artifact)*. doi:10.5281/zenodo.19045151
- [44] Roland Wismüller. 1994. Debugging of globally optimized programs using data flow analysis. In *Proceedings of the ACM SIGPLAN 1994 conference on Programming language design and implementation*. 278–289. doi:10.1145/178243.178430
- [45] Xiaodong Yu, Fengguo Wei, Xinming Ou, Michela Becchi, Tekin Bicer, and Danfeng Yao. 2020. GPU-based static data-flow analysis for fast and scalable android app vetting. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 274–284. doi:10.1109/IPDPS47924.2020.00037
- [46] YunaiV. 2026. RuoYi-Vue-Pro: A Modular Spring Boot Business Management System. <https://github.com/YunaiV/ruoyi-vue-pro>. Accessed: Jan. 2026.
- [47] Chen Zhang, Peng Li, Guangyu Sun, Yijin Guan, Bingjun Xiao, and Jason Cong. 2015. Optimizing FPGA-based accelerator design for deep convolutional neural networks. In *Proceedings of the 2015 ACM/SIGDA international symposium on field-programmable gate arrays*. 161–170. doi:10.1145/2684746.2689060
- [48] Zheng Jie. 2025. EL-ADMIN: A Web-based Enterprise Management System. <https://github.com/elunez/eladmin>. Accessed: Jan. 2026.
- [49] Zhiqiang Zuo, Rong Gu, Xi Jiang, Zhaokang Wang, Yihua Huang, Linzhang Wang, and Xuandong Li. 2019. BigSpa: An efficient interprocedural static analysis engine in the cloud. In *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 771–780. doi:10.1109/IPDPS.2019.00086
- [50] Zhiqiang Zuo, Yiyu Zhang, Qiuhong Pan, Shenming Lu, Yue Li, Linzhang Wang, Xuandong Li, and Guoqing Harry Xu. 2021. Chianina: An evolving graph system for flow-and context-sensitive analyses of million lines of C code. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 914–929. doi:10.1145/3453483.3454085

Received 2026-03-17; accepted 2026-08-06